

The Danger of Convenience: Unveiling the Explosively Amplified Security Risks Posed by Third-Party JavaScript Inclusion and Caching

SHENGPING BI, University of North Texas, United States

LANG ZHOU, University of North Texas, United States

TAO WANG, University of North Texas, United States

TAO HOU, University of North Texas, United States

MINGKUI WEI, George Mason University, United States

ZHUO LU, University of South Florida, United States

YAO LIU, University of South Florida, United States

SONG FANG, University of Oklahoma, United States

Today's web applications feature the proliferation of third-party JavaScript inclusion, which incurs a range of security risks. Although attack strategies by manipulating third-party JavaScript files have been widely investigated, the adverse impact caused by third-party JavaScript inclusion and caching does not receive much attention. Specifically, when a malicious script is cached, it can revive and bite every time when a user visits any website that includes it, leading to a much worse effect of the attack. In this article, we present the first comprehensive study on Alexa top-1 million websites to investigate how likely third-party JavaScript inclusion and caching can make an attack large-scale and long-lasting, and further uncover insecure practices that carelessly or inadvertently exacerbate the attack impact. We develop a customized toolkit on top of Chromium and build three tree structures to capture the complex inclusion relationships between websites and third-party JavaScript files. Meanwhile, we propose a time bounding technique to accurately estimate the caching duration of a JavaScript file. The results of our study reveal that a large number of websites overlook the adverse impact and are careless about the third-party JavaScript inclusion and caching control settings. We also discuss potential solutions to improve the current practices to minimize the security risk associated with third-party JavaScript inclusion and caching.

CCS Concepts: • **Security and privacy** → **Web application security; Vulnerability management; Web protocol security; Software and application security**; • **Information systems** → *Web applications; Web services; World Wide Web.*

An earlier version of the work [1] was presented in IEEE Conference on Communications and Network Security 2022.

Authors' Contact Information: Shengping Bi, Computer Science and Engineering, University of North Texas, Denton, TX, USA; e-mail: shengpingbi@my.unt.edu; Lang Zhou, Computer Science and Engineering, University of North Texas, Denton, TX, USA; e-mail: langzhou@my.unt.edu; Tao Wang (corresponding author), Computer Science and Engineering, University of North Texas, Denton, TX, USA; e-mail: tao@unt.edu; Tao Hou (corresponding author), Computer Science and Engineering, University of North Texas, Denton, TX, USA; e-mail: tao.hou@unt.edu; Mingkui Wei, George Mason University, Fairfax, VA, USA; e-mail: mwei2@gmu.edu; Zhuo Lu, University of South Florida, Tampa, FL, USA; e-mail: zhuolu@usf.edu; Yao Liu, University of South Florida, Tampa, FL, USA; e-mail: yaoliu@ieee.org; Song Fang, University of Oklahoma, Norman, OK, USA; e-mail: songf@ou.edu.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2471-2566/2026/11-ART37

<https://doi.org/10.1145/3833382>

Additional Key Words and Phrases: Web security, third-party javascript, browser caching, security analysis, insecure practices

ACM Reference Format:

Shengping Bi, Lang Zhou, Tao Wang, Tao Hou, Mingkui Wei, Zhuo Lu, Yao Liu, and Song Fang. 2026. The Danger of Convenience: Unveiling the Explosively Amplified Security Risks Posed by Third-Party JavaScript Inclusion and Caching. *ACM Trans. Priv. Sec.* 29, 4, Article 37 (November 2026), 29 pages. <https://doi.org/10.1145/3833382>

1 Introduction

It has become a common and convenient practice for a website to include third-party JavaScript libraries on the Internet. Many service providers, such as jQuery [2], Google Analytics [3], and various advertising and social network platforms, create and provide third-party JavaScript libraries for the Internet to use. As a result, web developers can reduce their development time and enrich their websites by simply including the third-party JavaScript libraries in their websites to provide extended functionalities, such as form submission, interactivity, animations, widgets, and visit activity tracking. To further improve performance, many of these scripts are hosted on high-availability infrastructure such as **Content Delivery Networks (CDNs)**, enabling websites to achieve reduced latency, faster loading times, and lower bandwidth demands on their own servers. However, loading third-party scripts is potentially dangerous as they gain the same privileges as local scripts of the webpage during the execution [4], which yields to a range of web attacks, such as **cross-site scripting (XSS)** [5, 6], **cross-site request forgery (CSRF)** [7], and **cross-site script inclusion (XSSI)** [8].

Typically, resources from different websites should be isolated from each other to avoid potential cross-site attacks. Nevertheless, if two websites include the same third-party scripts, they become correlated to each other. Consequently, they will face the same risks if the third-party scripts are exposed to attacks mentioned above. Further, such a practice produces an inadvertently negative impact from the security perspective when it meets the browser caching mechanism, which saves previously fetched web resources at local storage to accelerate website loading performance. In particular, successfully compromising a third-party script is usually a one-shot deal and only limited websites may be affected. But if the compromised script is cached, it can be loaded locally whenever a user visits any website that includes the same script. In this way, caching such a script can in fact make the attack impact both large-scale and long-lasting.

For example, Alice goes to a coffee house and connects to a free public WiFi network, which is set up by Bob, an attacker. Alice knows that a public free WiFi may be insecure, and thus she visits websites cautiously and avoids inputting sensitive information like her password. Nevertheless, if Bob is able to impersonate the third-party JavaScript sever and sends a modified, malicious third-party JavaScript file to Alice, Bob can generate a post-attack effect. Specifically, this malicious script will be cached by the browser. The next day, when Alice connects to a secure network and inputs her password to any website that includes the same script. The cached malicious script will revive and steal the password from Alice. While such in-transit manipulation scenarios have become less feasible due to the widespread deployment of HTTPS and **HTTP Strict Transport Security (HSTS)**, the risks analyzed in this work are not limited to MITM attacks. In practice, malicious scripts may also be introduced through compromised third-party providers, CDN infrastructure, or software supply-chain attacks. Once introduced, these scripts can be cached and reused across multiple websites, amplifying their impact.

This example shows that third-party JavaScript inclusion and caching opens a door for indirectly compromising a website. However, existing research mainly focuses on attack strategies that take

advantage of the vulnerability of JavaScript to successfully compromise users' privacy [9–13]. The adverse impact caused by third-party JavaScript inclusion and caching seems not receiving as much attention. Nevertheless, such an impact should not be left untended, but be contained or minimized. According to our study on Alexa top-1 million websites, we find that as many as 824,290 websites include third-party scripts, and a third-party script can be included in up to 504,692 websites. This indicates that the aforementioned attack could be common for Internet surfing. Further, our study reveals that 51.04% third-party scripts are set to cache for more than 100 days, indicating that this attack may impose a long-lasting effect on victim users.

Hence, rather than presenting a particular attack against a third-party script, the goal of this article is to investigate security issues regarding how likely third-party JavaScript inclusion and caching can make such an attack large-scale and long-lasting, and further to uncover insecure practices that carelessly or inadvertently exacerbate the attack impact. In this article, we provide the first comprehensive Internet-wide study to evaluate current practices of third-party JavaScript inclusion and caching from two perspectives:

- **Attack Scale:** Websites are becoming heavily correlated due to inclusion of widely-used third-party scripts. The transitivity of the inclusion (i.e., a script may include another script, which further includes more) may complicate the correlations between websites. It is essential to track each third-party script of websites to quantitatively understand how current inclusion practices can inadvertently increase the potential scale of an attack launched from a cached script.
- **Risk Duration:** Caching is a mechanism primarily designed for efficiency but not for security. As caching may allow a malicious script to survive in a local browser for quite a while, it is necessary to revisit current Internet practices of setting up the duration of third-party JavaScript caching from the security perspective.

In this study, we meet following challenges and propose corresponding techniques to address them.

- (1) A website may indirectly include a third-party script (e.g., a website includes a third-party script J_1 , which further includes another one J_2). Indirectly included scripts complicate the inclusion relationships. We must be able to identify all indirectly included scripts and track the inclusion relationships between different scripts. To solve this, we develop a customized, automated toolkit on top of Chromium [14] that can intercept the requests and responses between clients and servers, and exactly track the scripts that initiate each request. Further, we define three different tree structures to capture the inclusion relationships between websites and scripts.
- (2) When a third-party script expires in the cache, the browser needs to check whether the script on the server is updated or not. If updated, the browser fetches the new version from the server, otherwise it continues using the old script. Therefore, rather than just evaluating the expiration time of the script, it is necessary to measure its life time, which is defined as the time duration that a script remains unmodified on the server. However, it's difficult to exactly predict when a script will be updated in the future. We propose a time bounding technique that can accurately bound the life time to solve this.
- (3) We must be able to identify each third-party script from all crawled scripts. Intuitively, since third-party scripts are usually included from different domains with the website, we may identify them by simply comparing domain names of scripts and websites. This method, however, may not always return correct identification results, because a website may maintain its local scripts in an independent server, which also has a different domain name from the

website. To address this, we build a host list to first identify the domain names of third-party service providers. Then, we can identify each third-party script if their domain names match any entry in the list.

We study the third-party JavaScript inclusion and caching of Alexa top-1 million websites by collecting all associated scripts and caching configurations in HTTP headers.¹ We find that the Internet has largely overlooked the risk of caching third-party scripts and identify careless, insecure practices. Our major findings can be summarized as follows.

- **Complicated and insecure inclusion relationships:** We find that indirect inclusion is in fact overlooked by many website developers in their designs. Specifically, we find that 89,723 websites relocate their directly included third-party scripts on local servers, such that browsers can load these scripts from their servers to reduce security threats, but as many as 13,746 (15.32%) of them neglect to store at least one indirectly included third-party script on local servers. The results indicate that the complicated inclusion relationships between websites and scripts increase the management difficulty of third-party scripts, and a portion of websites indeed overlook to relocate all indirectly included third-party scripts.
- **Inconsistent configurations of Expires and max-age:** HTTP 1.0 and HTTP 1.1 propose metrics Expires and max-age to set the expiration time of a resource in a browser, respectively. Nevertheless, we find that 32.54% third-party scripts have inconsistent Expires and max-age, causing the expiration time computed from the two metrics being different from each other. The results indicate that web developers may be less aware of the security risk of caching and careless when setting these metrics.
- **Unawareness of security related settings:** HTTP 1.1 defines multiple security-related metrics to refine the behavior of JavaScript caching in Cache-Control (e.g., no-cache requires that a cached file cannot be used until the browser validates that it is not updated on the server). Nevertheless, our study reveals that 81.33% third-party scripts do not configure any security setting to improve the caching security.
- **Extremely long life time:** A well-maintained third-party script should be updated in a timely manner. Nevertheless, we find that 43,665 third-party scripts have life time larger than 720 days, indicating that these scripts have not been updated for at least two years. Because the browser reuses the stale cached script unless it is updated on the server, extremely long life time settings can advance the adverse impact of the attack launched from a cached third-party script.
- **Risk-incurring expiration time configurations:** HTTP 1.1 suggests that the expiration time should be set to a relatively small fraction of the resource life time (a commonly referenced guideline is around 10% [15]). However, our study reveals that the expiration time of at least 10.65% of third-party JavaScript files exceeds their life time by an order of magnitude. Such configurations may increase the persistence of stale or compromised scripts in the browser cache, thereby amplifying the potential impact of third-party JavaScript inclusion and caching.

Extension from Prior Work. This article extends our prior work published at IEEE CNS 2022 with significantly expanded scope and analysis. First, we enhance the measurement study with a more comprehensive characterization of third-party JavaScript inclusion structures, introducing new metrics to capture cross-website dependency relationships. In particular, we develop new analytical representations, including the JavaScript Backtracking Tree and the Website Correlation Tree, to

¹Note that HTTPS is a version of secure HTTP communication and has the same header information. We use the HTTP header to indicate either HTTP or HTTPS header unless otherwise specified throughout this article.

systematically quantify large-scale script propagation across websites. Second, we refine the temporal analysis of caching behavior by introducing improved time-based metrics that better capture script persistence and update dynamics. Third, we provide a more detailed analysis of deployment practices, including caching configurations and large-scale script reuse patterns, and examine their associated security implications. Finally, we strengthen the overall study by improving the methodology, expanding the measurement analysis, and incorporating a dedicated discussion of limitations and practical considerations. These extensions provide a more comprehensive understanding of the large-scale risks associated with third-party JavaScript inclusion and caching.

The remainder of this article is organized as follows. In Section 2, we introduce the necessary preliminaries. In Section 3, we describe the adverse impact of third-party JavaScript inclusion and caching, and outline our research objectives. Sections 4 and 5 present the research methodology and experimental setup, respectively. Section 6 presents and analyzes the measurement results. We discuss the limitations of our study and provide additional insights in Section 7. Potential solutions and mitigation strategies are discussed in Section 8. Finally, we review related work in Section 9 and conclude the article in Section 10.

2 Preliminaries

In this section, we briefly describe the background knowledge of third-party JavaScript, and web browser caching.

2.1 Third-Party JavaScript and Security

Client-side JavaScript has been extensively used in modern web applications as it allows source code to be executed directly within the client's browser, thereby reducing server-side workloads and enabling fast, responsive user interactions. To accelerate development and improve performance, it has become a common practice for websites to include third party JavaScript libraries. These libraries, provided by external vendors such as Google, Facebook, and various analytics or advertisement platforms, offer prebuilt functionalities like form validation, user tracking, and interactive visualizations. Developers can integrate them quickly using `<script>` tag to include a third-party script.

One of the main reasons for this widespread adoption is performance optimization. Hosting scripts on third-party servers (e.g., an CDNs) helps reduce latency and offload network traffic from the website's own servers. Additionally, caching mechanisms at the browser level enable the reuse of previously fetched scripts, resulting in faster page loads and reduced server-side bandwidth usage.

However, this convenience comes at the cost of increased security risks. Because third party scripts execute with the same privileges as a website's local scripts, any compromise to these external resources can have severe consequences. Despite their popularity, third-party JavaScript may incur a range of security issues [5, 6, 8, 16–23]. Further, if an attacker is able to inject malicious codes into the cached script, the manipulated script can be executed whenever a user accesses any website including the script. This, in turn, enables the attacker to launch attacks of password interception [24], privacy tracking [25], and phishing [26].

Moreover, since these scripts are maintained externally, timely updates and security validations are not always guaranteed. Developers may lack visibility or control over whether the scripts have changed, let alone whether they contain vulnerabilities. Practices such as isolated caching or integrity checks (e.g., **Subresource Integrity (SRI)**) are often neglected because they introduce operational complexity or require frequent script validation. In effect, performance and cost-saving incentives inadvertently discourage secure caching practices and centralized update mechanisms.

This tradeoff between performance and security can lead to large-scale and long-lasting impacts when cached third party scripts are compromised. Our work aims to quantify and raise awareness of these under-appreciated risks.

2.2 Browser Caching Policy

Fetching web content over the Internet can be time-consuming, especially when dealing with large resources such as JavaScript files, images, and multimedia content. Repeatedly downloading the same resources for each page visit not only increases loading latency but also places unnecessary strain on both client-side bandwidth and server-side infrastructure. To address this challenge, modern web browsers implement sophisticated caching mechanisms that store previously fetched content in local storage. These cached resources are reused during subsequent visits, allowing browsers to bypass network requests for unchanged content. This approach significantly reduces page load times, improves user experience, and decreases overall network traffic. Resource caching has become a foundational performance optimization in today's web architecture, and it is supported by all major browsers through standardized protocols and cache control headers [27].

2.2.1 Cache Indexing Mechanism. In practice, modern browsers use the full URL of a web resource as the cache key when storing and retrieving cached content. This means that a cached JavaScript file will be reused across websites as long as they reference the same exact URL, including the protocol, domain name, path, and query parameters. For example, if multiple websites include a script from `https://cdn.example.com/lib/script.js`, the browser will load the file from its local cache after the first visit, without re-fetching it from the server unless the cache has expired or validation is required. This default behavior significantly improves performance but also introduces the risk that a compromised script can be silently reused across many websites, especially when the same third party URL is broadly shared.

2.2.2 Cache Management Metrics. Browser cache management determines when a cached resource should be reused or revalidated to ensure content freshness. While users can manually clear caches, caching behavior is primarily controlled by HTTP headers, which can be broadly categorized into client-side and server-side metrics.

- *Client-side metrics:* These metrics specify how long a resource can be cached locally. The most relevant headers include `Cache-Control` and `Expires`. In practice, `Cache-Control` is widely used, with `max-age` defining the duration for which a resource is considered fresh. Once this duration expires, the browser must revalidate the resource with the server. Other directives (e.g., `no-cache` and `no-store`) further control whether cached content can be reused or must be revalidated, but are less central to our analysis.
- *Server-side metrics:* Server-side validation is typically performed using `ETag` and `Last-Modified`, which allow the browser to check whether a cached resource has changed. Upon expiration, the browser sends a conditional request, and the server responds with either an updated resource or a “304 Not Modified” response. These mechanisms determine how cached scripts are refreshed over time.

3 Understanding the Adverse Impact

In this section, we first present the attacker model to illustrate the security risks associated with third-party JavaScript inclusion and caching. We then outline the primary research objectives addressed in this article.

3.1 Attack Model

JavaScript caching indeed reduces webpage loading time; on the other hand, however, it incurs the security concern of attacks across multiple websites. As shown in Figure 1, `sample.js` at a third-party service provider is included in websites $1 \sim n$. When a user visits website 1 for the first time, `sample.js` is retrieved from the third-party service provider and cached in the user's local storage. When the user subsequently visits any of the websites $1 \sim n$, the browser will simply load

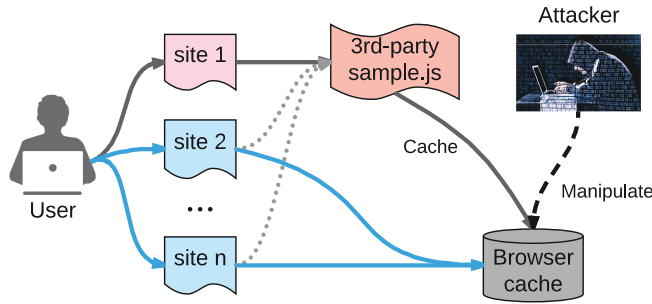


Fig. 1. The adverse impact of third-party JavaScript inclusion and caching.

`sample.js` from the local cache unless the cached script expires. If `sample.js` is manipulated by an attacker when the user visits website 1, the browser will run the cached, compromised `sample.js` during the subsequent visits to any of the websites 1~ n .

The strategies of the initial attack that can comprise the scripts has been widely investigated in the security community [10, 12, 13]. In particular, such attacks include client-side attacks that utilize vulnerabilities existing in a browser to access cached files and modify their contents, such as magecart attacks [13, 28], server-side attacks that try to compromise the script hosted in the third-party server, such as third-party domain re-registration [10], CDN hijacking [29], NPM package poisoning [30], and direct host compromising [12], and in-transit attacks that inject malicious scripts by exploring the communication vulnerabilities between a browser and a server [9], such as **man-in-the-middle (MITM)** attack using rogue WiFi APs [11]. An attacker can force the refresh of a cached script and deliver a malicious version of the script (e.g., using `<new Image.src="URL/to/malicious.js">`). Recent research [31–34] also reveals that when HTTPS is used, although modern browsers always warn users of invalid certificates under in-transit attacks, more than 50% users still click through the warning to visit the corresponding websites. Moreover, people are more inclined to ignore warnings when they visit websites without privacy information, because they may perceive that there is no risk to visit such websites. However, as shown in Figure 1, a compromised script can be cached and come to bite later.

Once a malicious script is included and cached, it may persist across repeated visits and potentially affect multiple websites that reference the same resource. This persistence can amplify the attack surface and enable long-lasting, large-scale impact, even without continuous attacker involvement. This model highlights how performance-oriented mechanisms in modern web infrastructure, particularly caching and reuse of shared scripts, can introduce structural security risks. We note that modern browser defenses such as cache partitioning may reduce cross-site cache reuse in certain scenarios; we discuss these aspects in detail in Section 7.

3.2 Research Objectives

The attack model demonstrates the potential attack impact of third-party JavaScript inclusion and caching, which consists of two key components: (i) the inclusion relationships between websites and scripts; (ii) the caching settings of scripts in local storage.

The website-JavaScript inclusion relationships and caching settings are largely determined by the current Internet practices. The website-JavaScript inclusion relationships have a scale consequence of an attack. For example, a large number of websites can be vulnerable when they include the same malicious script (e.g., a large n in Figure 1). Caching settings cause a time consequence of an attack. For example, a very long caching duration of a malicious script makes the impact of an attack long-lasting.

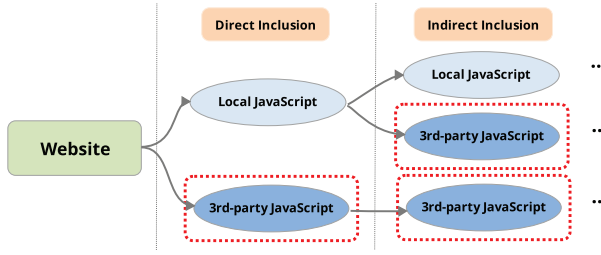


Fig. 2. Categories of JavaScript files in a website.

Although such negative consequences depend on the success of an initial attack, we can never focus solely on defending against the initial attack and ignore the fact that inclusion relationships and caching settings may further advance the attack. Rather, it is necessary to revisit the current Internet practices, and to uncover and improve insecure ones to limit or minimize the scope and impact of a successful initial attack.

To this end, we aim to propose strategies to enable an Internet-wide study. Our objectives are twofold: (i) to quantitatively and systematically evaluate current practices of third-party JavaScript inclusion and caching from both scale and time perspectives; and (ii) uncovering insecure practices that inadvertently increase the scale or time, thereby escalating the impact of a successful initial attack.

4 Methodology

In this section, we first classify JavaScript files based on how they are included, and then present three tree-based structures to systematically characterize the relationships between websites and JavaScript files. We further describe our methods to quantify the website-script inclusion relationships and the caching effects from both scale and time perspectives.

4.1 JavaScript Classification

The inclusion relationships between websites and third-party scripts are complicated on today's Internet. For example, a webpage can include a script, which also includes multiple scripts that further include more. As shown in Figure 2, we classify scripts included in a website based on two properties: (i) local or third-party scripts, depending on where scripts are hosted; (ii) directly or indirectly included scripts.

Our study focuses on third-party scripts, as highlighted in Figure 2. In what follows, we describe how to track all directly and indirectly included third-party scripts according to their inclusion relationships.

4.2 Inclusion Relationships

As shown in Figure 2, a JavaScript file may be included directly or indirectly by a website. Because of indirect inclusion, the website may contain multiple levels of recursive inclusions among scripts. As long as a third-party script is included in any level, the website may be exposed to the security risk of caching a compromised version of the script. We propose three tree structures to characterize the inclusion relationships between scripts and websites.

4.2.1 JavaScript Inclusion Tree. For each website, we use the depth-first search algorithm to construct the JavaScript Inclusion Tree, which fully characterizes how scripts are included in a given website. In particular, we specify a website as the root; all scripts included directly by the website are added as children of the root; then for any script pair (J_1, J_2) with J_1 including J_2 , we add J_2 as

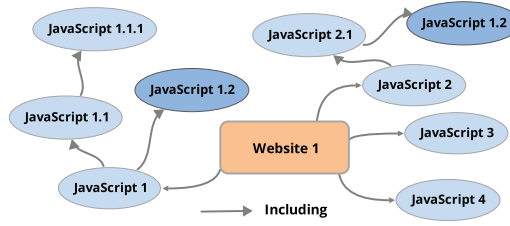


Fig. 3. Example of JavaScript inclusion tree with Website 1 as the root.

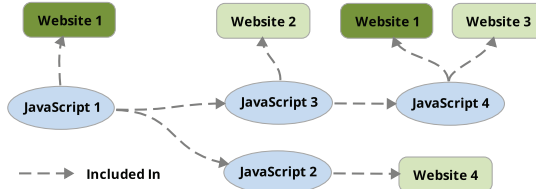


Fig. 4. Example of JavaScript backtracking tree with JavaScript 1 as the root.

a child of J_1 . Figure 3 shows an example of the JavaScript Inclusion Tree for a given Website 1. Note that if the same script is included in different scripts (e.g., JavaScript 1.2 is included in both JavaScript 1 and 2.1 in Figure 3), it may appear multiple times in the tree. We eliminate any duplicates in measurement evaluations.

4.2.2 JavaScript Backtracking Tree. For each third-party script, we use a backtracking algorithm to build the JavaScript Backtracking Tree, which traces how a third-party script has been included in different websites. In particular, we specify a given script as the root. All the websites and scripts directly including it are added as children of the root. For any child as a script in the tree, we further add all websites and scripts directly including it as its children, and so on. The JavaScript Backtracking Tree is constructed recursively and is complete when all the leaves of the tree are websites. Figure 4 gives an example of the JavaScript Backtracking Tree of a given JavaScript 1. Note that a website (e.g., Website 1 in Figure 4) may appear multiple times as leaves in the JavaScript Backtracking Tree; we also eliminate any duplicates during measurement evaluations.

The JavaScript Backtracking Tree characterizes all the websites associated with the same security risk of third-party JavaScript caching. For example, suppose that when a user visits Website 1 that includes JavaScript 1 (as shown in Figure 4), a compromised version of JavaScript 1 was cached in the user's browser; then, the user is still vulnerable when he/she visits Website 2 that indirectly includes JavaScript 1 via JavaScript 3.

4.2.3 Website Correlation Tree. For each website, we also build the Website Correlation Tree to illustrate how a website is correlated with others in terms of the third-party JavaScript caching vulnerability. Specifically, two websites are considered as correlated if they directly or indirectly include at least one identical third-party script. A Website Correlation Tree is built upon previous two tree structures. In particular, given a website as the root, we first build the JavaScript Inclusion Tree upon the root. For each non-duplicated script as a leaf in the tree, we expand the leaf with its JavaScript Backtracking Tree. Note that we do not expand non-leaf nodes, because the JavaScript Backtracking Tree of the leaf nodes must include the non-leaf nodes.

An example is shown in Figure 5. Website 1 includes JavaScripts 1~3. Through building backtracking trees for the three scripts, Websites 2~7 are added as leaves on the tree. Similar to previous

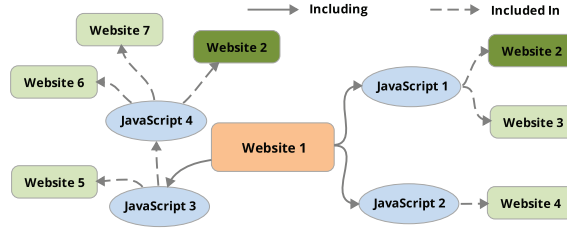


Fig. 5. Example of website correlation tree with Website 1 as the root.

trees, a website (e.g., Website 2 in Figure 5) may appear multiple times as leaves on a Website Correlation Tree. We also eliminate the duplicates during measurement evaluations.

4.3 Obtaining Scale Metrics from Trees

After introducing the three tree structures to fully capture the inclusion relationships between websites and scripts, we can extract the scale metrics from the trees to understand the adverse impact of third-party JavaScript inclusion and caching. In what follows, we define three major scale metrics and describe methods to obtain these metrics from the trees.

- *Number of scripts included in a given website:* This metric is defined as the number of third-party JavaScript files included either directly or indirectly in a given website. A website may be exposed to the cached third-party JavaScript vulnerability, as long as it includes the third-party script at any level in its JavaScript Inclusion Tree. The more the number of scripts included in a website is, the more vulnerable it can be.

The metric is measured via the JavaScript Inclusion Tree. In particular, for a given website, we can build its corresponding JavaScript Inclusion Tree to track all the third-party scripts directly or indirectly included. The metric is computed by counting the number of non-duplicate nodes in the tree.

- *Number of websites including a given script:* This metric is defined as the number of websites that either directly or indirectly include a given JavaScript file. As discussed previously, when a cached third-party script has been compromised, visits to all the websites including it become vulnerable. Thus, given a third-party script, we use this metric to understand the scale of the potential vulnerable websites on the Internet when it is compromised and cached. The metric is measured from the JavaScript Backtracking Tree. Specifically, for a given third-party script, its JavaScript Backtracking Tree traces back all the websites that include it. The value of the metric equals to the number of non-duplicate leaves of the tree.
- *Number of websites correlated to a given website:* Given a website, this metric is computed based on its Website Correlation Tree. In particular, we build the Website Correlation Tree associated with the given website, and then count the number of all non-duplicate leaves of the tree.

This metric indicates the maximum number of websites that are correlated to a given website with a potential JavaScript caching vulnerability.

4.4 Obtaining Time Metrics

Time metrics present another dimension of the security impact associated with third-party JavaScript inclusion and caching. When a compromised JavaScript file is cached, it remains harmful unless the cache is refreshed. As a result, we also aim to investigate the caching duration for a script to understand the current practices on the Internet.

As discussed in Section 2.2, the caching duration of a script is influenced by both client-side metrics (e.g., Expires and max-age) and server-side metrics (e.g., ETag and Last-Modified). Client-side metrics define how long a cached script is considered fresh and can be reused without revalidation. Server-side metrics support conditional requests, allowing the browser to verify whether a resource has been updated when revalidation occurs. Intuitively, a compromised script can survive in the cache until it Expires or reaches the max-age, because the browser only revalidates the cached script when it expires (i.e., to send a request to check if it is modified on the server). It seems that the caching duration can be evaluated based on client-side metrics Expires and max-age only.

However, this is not always the case: (1) An expired script may still be used after revalidation with the third-party server. In particular, when a cached script expires, the browser sends an HTTP request with the previously stored Last-Modified and ETag of the script to the server. The server then checks current Last-Modified and ETag. Many times, the script is still not changed and the server returns a “304 Not Modified” response. Upon receiving the response, the browser continues to keep the cached script in local storage and resets its Expires or max-age; (2) If a powerful MITM attacker is able to temporarily control the entire connection between a client and a server [9], it can modify Expires or max-age in the HTTP headers to any arbitrary value to maximize the caching duration. Nevertheless, a large Expires or max-age does not always indicate a long caching duration. A user may manually make a hard refresh to enforce the revalidation, and a web developer may also configure the script to force the revalidation.

Therefore, both client-side metrics (Expires and max-age) and server-side metrics (Last-Modified and ETag) are vital to indicate the caching duration of the script. We define two time metrics to evaluate the temporal impact.

- *Local caching time of a script*: This metric, which is also referred to expiration time, indicates the time duration a script is cached in a browser’s local storage until the browser sends a request to the server check if it is modified. The value of local caching time equals max-age in HTTP 1.1 or the time interval from the measurement time to Expires in HTTP 1.0.
- *Life time of a script*: This metric indicates the time duration that a third-party script remains unmodified in the server. Intuitively, if a script is not modified frequently on the server, it can be cached in the local storage of a browser for quite a while even with a small value of local caching time, because the server always sends “304 Not Modified” responses. We measure the life time as the time interval between two consecutive changes of Last-Modified/ETag of a script.

4.4.1 Bounding Life Time. Local caching time of a script can be easily measured from the metrics max-age and Expires. Nevertheless, as a script may remain valid on a server for hours, days, years, or even never be modified, it is challenging to capture two exactly consecutive changes of Last-Modified/ETag to obtain the life time. We propose a time bounding technique to accurately bound the life time.

As a solution, we repeatedly crawling and estimate upper and lower bounds of the life time of each script. Particularly, we record the Last-Modified/ETag for each third-party script during our first measurement, and try to find the change of the Last-Modified/ETag in our subsequent measurements. Suppose that for a third-party script, we first capture the change of its Last-Modified/ETag in the k th measurement. Then, the lower bound of the life time of the script is the time interval from its Last-Modified in the first measurement to the $(k - 1)$ th measurement time. As shown in the timeline of JavaScript 1 in Figure 6, we record Last-Modified as $JS1.LM1$ during the first measurement, and find it changes to $JS1.LM2$ during the second measurement. Then, the lower bound of its life time is the first measurement time minus $JS1.LM1$. Because we can be sure that it is not modified from $JS1.LM1$ to the first measurement time.

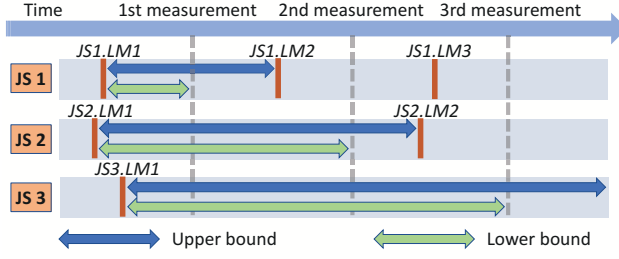


Fig. 6. Examples: bounding the life time.

The upper bound of life time of scripts is computed as the time interval from its Last-Modified in the first measurement to the Last-Modified, that is, observed to be changed for the first time in the subsequent measurements. For example, JavaScript 1 in Figure 6 has the first observed change of Last-Modified from *JS1.LM1* to *JS1.LM2* in the second measurement; then the upper bound of its life time is $JS1.LM2 - JS1.LM1$.

5 Web Data Crawling and Third-Party JavaScript Identification

After describing the methods of the scale and time evaluations, we create the toolkit to set up our web data crawling experiments. In what follows, we describe the toolkit as well as our strategy to identify third-party scripts, which is critical for the measurement and evaluation.

5.1 Web Data Crawling

We target crawling Alexa top-1 million websites, which include most popular websites on the Internet. We develop a customized, automated toolkit to collect JavaScript files and corresponding HTTP headers. This toolkit is developed on top of Chromium [14] utilizing the Chrome DevTools Protocol [35]. A high-performance workstation is used for web data crawling, which equips with dual Intel Xeon Gold processors and 64 GB memory. The running operating system is CentOS.

The toolkit creates multiple parallel processes to speed up the crawling. It runs as a headless browser, and simulates the visit to a target website as a real user [36, 37]. The headless model gives us a real browser context without the memory overhead of graphical user interface [38]. For each website, we set a timeout of 60 seconds to limit the total crawling duration, ensuring that the crawler does not get stuck when encountering network delays or unstable connections. To avoid excessive delays caused by individual resources, we also apply a 5-second timeout for each JavaScript file. If a script takes longer than 5 seconds to load, it is skipped to allow the overall crawl to proceed efficiently. The toolkit only crawls the homepage of each website since log-in is usually required for a deeper traversal. In addition, as our objective is to collect all the third-party scripts and their caching settings, other resource files, such as images and style sheets, are ignored. Besides, we clear browser cache, cookies, and histories before crawling each website. This prevents cached or dynamically generated scripts from affecting the results. All the HTTP connection details between client and server are recorded, including the associated cache settings for each script. The crawled data is stored in **HTTP Archive (HAR)** [39] format for further analysis.

It's challenging to collect all directly and indirectly included third-party scripts and capture their inclusion relationships. Simply parsing the content of a webpage is not efficient, and may not return all the included scripts, because an indirectly included or dynamically generated JavaScript file does not appear in the webpage directly (e.g., advertisement frameworks and iframes). To address this challenge, we utilize the Chrome DevTools Protocol to track all the HTTP transactions, including all the requests from the client and the responses from the server. By analyzing these

HTTP transactions, we can collect all the scripts, and further identify the initiator of each request and thus capture the inclusion relationships between scripts and websites.

We repeated our crawling on a weekly basis for 12 months, from May 1, 2021 to April 30, 2022, to obtain continuous and sequential measurements of JavaScript inclusion and caching settings. Among the Alexa Top-1 Million websites, we successfully collected data from 970,698 websites, excluding domains that were consistently unreachable or failed to respond during the crawling period. To evaluate whether the observed trends remain consistent in the current web ecosystem, we additionally conducted an updated crawl during the revision of this article in February and March 2026. Specifically, we collected data from 100,000 websites sampled from the original list of one million websites used in our initial study. To ensure representativeness across different popularity levels, the one million websites were divided into 100 ranking buckets, and 1,000 websites were randomly sampled from each bucket. This stratified sampling approach preserves the distribution of website popularity and reduces potential bias toward highly ranked domains. We applied the same crawling framework and analysis pipeline described above to this updated dataset and recomputed the key scale and time metrics to validate whether the main findings of our study continue to hold.

5.2 Third-Party JavaScript Identification

We design a method to identify whether a script is hosted on a local or third party server. A natural approach is to compare the domain name of the script with that of the website. If the domains match, the script is considered local; otherwise, it is considered third party. However, this heuristic can be misleading in certain cases. For instance, many websites with domain names like `www.example.com` include scripts from a different domain, such as `static.example*.com`, which may be mistakenly identified as third-party. Nevertheless, the resources on `static.example*.com` are actually created and used by the website `www.example.com` only.

Our analysis confirms that this pattern commonly appears in real-world websites. For example, `www.nytimes.com` includes scripts from `static01.nytimes.com`, and `www.cnn.com` loads scripts from `lightling.cnn.com`. Although these domain names differ, the resources are operated and maintained by the same organizations that manage the corresponding primary websites. Treating such scripts as third party would result in inaccurate classification and potentially overstate the extent of third party script usage and its associated security risks. To improve the accuracy of our measurements, it is important to identify and account for such cases, ensuring that these resources are correctly classified as local rather than third party.

To provide a more reliable mechanism to identify third-party scripts, we first build a host list to identify the domain names of third-party JavaScript service providers. Each entry in the list is the domain of a third-party JavaScript service provider. If the domain name of a script matches no entry in the list, it will be identified as local; otherwise, it will be identified as third-party.

We rank the domains based on their popularity, i.e., the number of websites that include any script from the domain. We refer to this number as *frequency of references*. A domain may host multiple scripts. If a single website includes multiple scripts from the same domain, it will be counted only once towards the frequency of references to that domain. A domain is added to the host list as long as its frequency of references is no less than 2. Table 1 shows the host list with top-10 frequencies of references from our crawling results. The web data is crawled from Alexa top-1 million websites, and therefore the host list is able to include most of the third-party JavaScript providers on the Internet.

To assess the accuracy of the host-list-based classification, we manually examined a stratified random sample of 100 host domains. For each domain, we inspected the associated websites, provider documentation, domain ownership information, and the scripts served by the domain to determine whether it represented an independently operated third-party service. All 100 sampled hosts were confirmed as third-party providers. This validation indicates that the host-list method is

Table 1. The 10 Most Popular 3rd-party Javascript Hosts

Rank	Third-party JavaScript Host	Frequency
1	www.google-analytics.com	601,717
2	connect.facebook.net	263,048
3	ajax.googleapis.com	190,621
4	pagead2.googlesyndication.com	171,373
5	www.googletagmanager.com	125,597
6	www.google.com	123,464
7	tpc.googlesyndication.com	100,913
8	www.googleadservices.com	92,661
9	platform.twitter.com	89,354
10	apis.google.com	86,353

reliable for the sampled providers, while uncertainty remains greater for low-frequency hosts that may be omitted from the host list.

6 Measurement Results and Analysis

With all measurement data, we build the proposed tree structures and compute the metrics from both scale and time perspectives. In this section, we first present the overall JavaScript usage statistics, and then evaluate the measured metrics to reveal the insecure practices.

6.1 Overall Usage of JavaScript

We successfully crawled 970,698 websites from Alexa top-1 million websites. From them, we collected 10,684,818 different scripts in total, in which 3,107,929 were identified as third-party ones. These third-party scripts have been included for 11,149,205 times. Nevertheless, we find that there are a large number, i.e., 2,778,633 (89.40% of 3,107,929), of third-party scripts are included only once. We call them *singletons*, and call the rest of third-party scripts *non-singletons*.

Observing a large number of singletons seems to be inconsistent with the intuition that third-party scripts should be shared across multiple websites. We find that these singletons are mainly due to two reasons: (i) upon the inclusion request from a website, a third-party provider may return the client a script with unique directory or filename dedicated to this website (e.g., pagead2.googlesyndication.com/pub-config/r20160913/ca-pub-7278577707988162.js); and (ii) a website developer or third-party provider may also append a unique query string to the URL of a script (e.g., cse.google.com/cse/cse.js?cx=017837193470827524476:s65nylv7hj0). Because the caching policy of a web browser identifies a script by its entire URL (including the query string), scripts with the same content but different domains, paths, file names, or query strings are indeed treated as different caching entities.

Singletons are included uniquely by their associated websites. In contrast, the rest 329,296 non-singletons are included 8,370,582 times. Each of these non-singletons is included in 25.42 websites on average, and 824,290 (84.92% of 970,698) websites include at least one non-singleton. We focus on non-singleton third-party scripts in our measurement and analysis, since singletons are not included across websites. In the following, when we mention third-party scripts, we mean non-singletons as they are widely included, unless otherwise specified.

6.2 Scale Metrics based on Tree Structures

We present measurement results on scale metrics based on proposed tree structures to demonstrate inclusion relationships between scripts and websites.

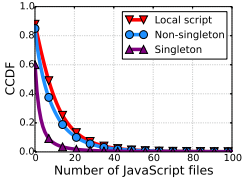


Fig. 7. Distribution of the number of third-party scripts per website.

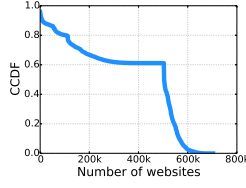


Fig. 8. Distribution of the number of correlated websites per website.

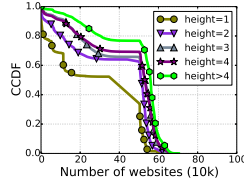


Fig. 9. Distribution of the number of correlated webs for different levels.

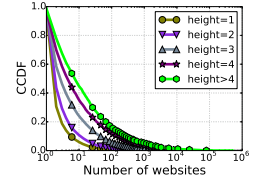


Fig. 10. Number of websites in different-level backtracking tree.

Table 2. Top-10 Third-Party Scripts Ranked by the Number of Websites Including Them

Rank	Third-party script	Number
1	www.google-analytics.com/analytics.js	504,692
2	pagead2.googlesyndication.com/..._impl.js	127,787
3	connect.facebook.net/en_US/fbevents.js	122,445
4	pagead2.googlesyndication.com/...google.js	120,620
5	pagead2.googlesyndication.com/.../osd.js	115,188
6	www.google-analytics.com/ga.js	113,782
7	tpc.googlesyndication.com/.../osd_listener.js	92,429
8	platform.twitter.com/widgets.js	82,681
9	tpc.googlesyndication.com/..._non_hydra.js	80,205
10	static.doubleclick.net/instream/ad_status.js	65,544

6.2.1 Third-Party JavaScript Deployment. We first investigate the third-party JavaScript deployment among websites.

- *Number of scripts included in a given website:* This metric is derived from the JavaScript Inclusion Tree and measures the number of third-party scripts included in a given website. Intuitively, a website including more third-party scripts exposes a higher security risk to potential attackers. According to our measurement, each website includes 19.29 scripts on average, and 8.62 of them are non-singletons. Figure 7 plots the **complementary cumulative distribution function (CCDF)** of the number of local scripts, singletons, and non-singletons included in each website. As in Figure 7, local scripts and non-singletons have similar distributions and are widely included, while singletons are less included.
- *Number of websites including a given script:* This metric is calculated from the JavaScript Backtracking Tree and describes the number of websites directly or indirectly including a given third-party script. This metric can indicate the maximum number of potential vulnerable websites if a particular script is manipulated. According to our measurement, a third-party script is included in 25.42 websites on average. Table 2 demonstrates the top-10 third-party scripts ranked by corresponding numbers of websites including them. As shown, a popular third-party script can be included in around 6% ~ 50% of Alexa top-1 million websites. (e.g. Google's analytics.js is included in 504,692 websites out of the one million websites).
- *Number of websites correlated to a given website.* Two websites are said to be correlated if they include at least one identical third-party script. By manipulating this script, an attacker can compromise both websites. The number is computed from the Website Correlation Tree,

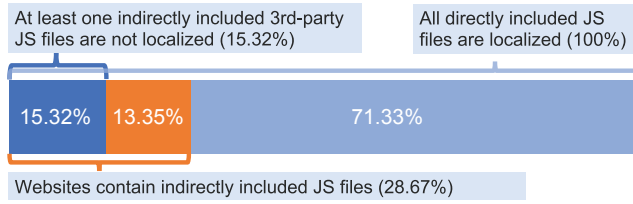


Fig. 11. Statistics about the localization of third-party JavaScript files.

and indicates the maximum number of websites, that are correlated to a given website with potential JavaScript caching vulnerabilities. We plot the CCDF of the number of correlated websites in Figure 8. It shows that 80.25% websites are correlated with more than 100,000 websites, indicating that the third-party JavaScript inclusion actually associates a substantial portion of websites.

6.2.2 Multi-level JavaScript Inclusion. Our measurement results show that indirectly included third-party scripts are common on the Internet. Specifically, according to our measurement, 551,689 websites contain indirectly included third-party scripts. In addition, all third-party scripts have been included for 8,370,582 times in total, and 3,068,041 inclusions among them are indirect. Our study further reveals that 56.84% websites have heights of JavaScript Inclusion Trees larger than 1, and the maximum height of the trees is surprisingly as large as 18. A website may correlate to more websites, when it has a higher JavaScript Inclusion Tree. Figure 9 shows the distribution of the number of correlated websites of a website given different heights of the JavaScript Inclusion Tree. The JavaScript Backtracking Tree tracks all the websites that directly or indirectly include a third-party script. In Figure 10, we plot the CCDF of the number of websites including a script given different heights of JavaScript Backtracking Tree. From these two figures, we can observe that both websites and third-party scripts tend to be correlated with or included in more websites as the height increases.

- **Insecure practice: careless localization of third-party JavaScript files.** Web developers may try to store all included third-party scripts on their local servers to reduce the security risk raised by third-party JavaScript inclusion and caching. During this practice, web developers must track all third-party scripts included in their websites. Nevertheless, we observe that a portion of web developers only pay attention to directly included third-party scripts but neglect indirectly included ones. In particular, as shown in Figure 11, we find that 89,723 websites store all their directly included third-party scripts on their local servers, but 13,746 (15.32% of 89,723) of them neglect to store indirectly included third-party scripts locally. This measurement results indicate that multi-level JavaScript inclusion makes the website security management complicated.
- **Insecure practice: limited adoption of SRI.** SRI is a W3C-recommended mechanism that enables browsers to verify the integrity of fetched scripts using predefined cryptographic hashes. Our measurements indicate that SRI remains relatively underutilized in practice: fewer than 5% of third-party scripts in our dataset are included with an SRI attribute. SRI is particularly well-suited for stable, versioned resources (e.g., libraries hosted on CDNs), where content remains unchanged and hashes can be maintained consistently. In contrast, its adoption for dynamically generated or frequently updated scripts is more challenging due to the overhead of maintaining and updating integrity hashes. As a result, many websites include remote scripts without integrity verification, which may increase exposure to compromised or modified third-party resources.

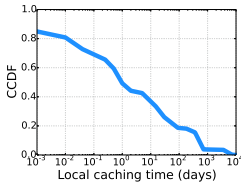


Fig. 12. Distribution of local caching time for different 3rd-party scripts.

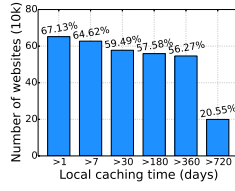


Fig. 13. Number of websites given different local caching time.

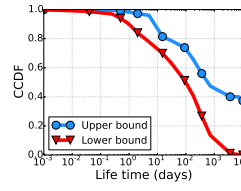


Fig. 14. Distribution of upper and lower bounds of life time of a script.

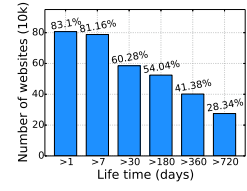


Fig. 15. Number of websites with scripts with life time > a threshold.

Table 3. Statistics of Security Related Settings

Cache-Control Settings	Count	Secure?
no-cache	18,180 (7.20%)	Yes
no-store	9,194 (3.64%)	Yes
must-revalidate	16,701 (6.61%)	Yes
immutable	5,043 (2.00%)	No

6.3 Time Metrics Measurement

In what follows, we evaluate time metrics based on the HTTP header information of third-party scripts.

6.3.1 Local Caching Time. As noted in Section 4.4, local caching time can be measured from the client-side metrics via max-age or Expires. Figure 12 illustrates the distribution of local caching time of third-party scripts. We find that over 115,517 third-party scripts are set to be cached in a browser for more than 10 days, and over 58,988 are set for more than 100 days. As a result, a malicious third-party script may survive in the cache for quite a long time. Figure 13 plots the number of websites containing at least one third-party script with local caching time in a given interval, and shows that as many as 57.58% websites include at least one script with local caching time larger than 180 days.

- **Insecure practice: inconsistent configurations of Expires and max-age.** Although max-age can override Expires when both metrics are presented in HTTP headers, they should be set consistent to avoid ambiguity, i.e., local caching time calculated from Expires and max-age should be equal to each other [15]. According to our results, 168,317 third-party scripts set both Expires and max-age. Nevertheless, we find that 54,771 scripts have inconsistent Expires and max-age. Moreover, 40,568 exhibit this difference larger than 50% of the value of max-age. These observations imply that web developers may be less aware of the security risk of caching and careless when setting these metrics.
- **Insecure practice: unawareness of security related settings.** HTTP 1.1 defines multiple security-related metrics to refine the behavior of JavaScript caching in Cache-Control (e.g., no-cache, no-store, immutable, and must-revalidate). According to our measurement, 252,529 third-party scripts configure max-age of Cache-Control. Nevertheless, 205,384 third-party scripts do not configure any security setting, such as no-cache, no-store and must-revalidate, to improve the security. Even worse, 5,043 scripts set the metric immutable to discourage the revalidation of the cached file with the third-party provider. Table 3 shows the number of third-party scripts that actually configure security related settings in Cache-Control.

6.3.2 Life Time. Next, we proceed to measure the life time of a third-party script, which is defined as the duration that a third-party script remains unchanged in the server. We use the estimation method proposed in Section 4.4 to bound the life time of a third-party script. Figure 14 plots the CCDF of upper and lower bounds of the life time for different third-party scripts. We see from Figure 14 that at least 241,769 third-party scripts have a life time larger than 10 days, and at least 168,072 have a life time larger than 100 days. We are also interested in the number of websites that include third-party scripts with long life time. Figure 15 shows the number of websites that include at least one third-party script with life time longer than a given threshold. We observe that over half of websites include at least one third-party script, whose life time is longer than half a year. This indicates that a substantial number of websites on the Internet include third-party scripts with long life time.

- **Insecure practice: extremely long life time settings.** A well-maintained third-party script should be updated in a timely manner. Nevertheless, Figure 14 shows that 43,665 third-party scripts have life time larger than 720 days, indicating that they have not been updated for at least two years. The possible reason can be that these third-party scripts are developed by unreliable providers or have been rarely maintained.

6.3.3 Local Caching Time vs. Life Time. HTTP 1.1 suggests that local caching time be set to a relatively small fraction of the resource life time (a commonly referenced guideline is around 10% [15]) to help ensure timely retrieval of updated content. We evaluate local caching behavior using the ratio between the configured local caching time and the upper bound of the resource life time. Under the above guideline, this ratio is expected to remain relatively small (e.g., around or below 10%). Note that, since we use the upper bound of the life time in our estimation, the computed ratio represents a conservative approximation.

- **Insecure practice: risk-incurring settings of local caching time.** Figure 16 illustrates the distribution of the ratios of all third-party scripts. We observe that about 29.23% scripts have ratios larger than 10%, and 10.65% scripts even have ratios larger than 1000%, indicating that their local caching time is improperly set on the Internet and does not conform the HTTP 1.1 suggestion. This further worsens the long-lasting impact of an attack.

While long cache durations are commonly adopted for performance optimization, especially for widely deployed scripts, such configurations may introduce security risks when the scripts are updated frequently or are not strictly versioned. In these cases, extended cache lifetimes can delay the propagation of updates, thereby prolonging the persistence of outdated or potentially compromised scripts and amplifying the impact of an attack.

6.4 Joint Analysis of Scale and Time

Previous measurement studies evaluate the adverse impact of third-party JavaScript inclusion and caching from the scale and time aspects separately. We observe that there are many third-party scripts included in a great number of websites, and also find that there are a substantial number of third-party scripts with very long local caching and life time. We wonder whether a widely-included script is also associated with a very long local caching or life time? To answer this question, it is necessary to combine the scale and time aspects together to analyze security impact of third-party JavaScript inclusion and caching.

Figure 17 box-plots the distribution of the local caching time of different third-party scripts as a function of the number of websites including them (measured from the JavaScript Backtracking Tree). We can see that the local caching time of a third-party script increases when it is included in more websites. For example, for third-party scripts included in more than 10,000 websites, their median local caching time is 365 days (most famous third-party service providers set 1 year as the local caching time, e.g., Google Hosted Libraries [40]).

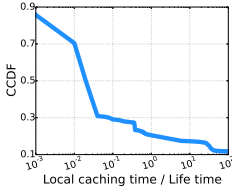


Fig. 16. Distribution of the ratio between local caching time and upper bound of life time.

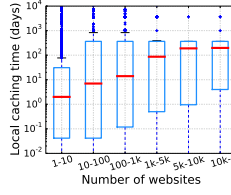


Fig. 17. Local caching time in different regions of the numbers of including websites.

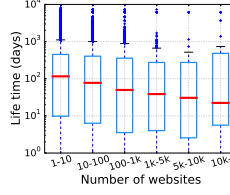


Fig. 18. Lower bounds under different regions of the numbers of including websites.

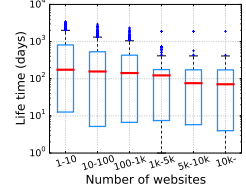


Fig. 19. Upper bounds under different regions of the numbers of including websites.

Figures 18 and 19 box-plot the distributions of the lower and upper bounds of the life time as a function of the number of websites including them, respectively. We can see that the lower and upper bounds of the life time of a third-party script both exhibit decreasing trends as it is included in more websites. This shows that popular third-party providers usually update their scripts in a frequent way. Nevertheless, as shown in Figures 18, most third-party scripts still have long life time for more than 50 days.

- **Insecure practice: setting local-caching time not in accordance to life time.** From a security perspective, scripts that are updated more frequently (i.e., with shorter effective life-times) may benefit from shorter cache durations (e.g., smaller Expires or max-age) to reduce the persistence of outdated or potentially compromised content. However, our measurements reveal a different trend. Comparing Figure 17 with Figures 18 and 19, we observe that more widely used scripts tend to be updated more frequently, while simultaneously being assigned longer cache lifetimes. While long cache durations are common for widely deployed scripts to improve performance, this mismatch between update frequency and caching policy may increase the window during which stale or compromised scripts remain active. Therefore, it is important to align caching policies with the update characteristics of scripts, particularly for non-versioned or dynamically changing resources, in order to mitigate potential security risks.

6.5 Website Number Given Scale and Time Thresholds

Intuitively, a website is easier to be affected by a cached malicious script if it is correlated with more websites and includes third-party scripts with longer caching time. We are interested in knowing how many websites on the Internet are correlated with a large number of other websites and at the same time include scripts with long caching time.

We use a trimmed Website Correlation Tree to count the number of such websites. Specifically, we first build the Website Correlation Tree of a given website. We trim the tree by deleting branches with scripts whose caching time (measured by local caching time or life time) is less than a given time threshold T_t . Then, we compute the number of leaves in the trimmed tree (i.e., the number of other websites correlated with the given website via a script with caching time greater than T_t). Figure 20 illustrates an example of a trimmed Website Correlation Tree: the branch with JavaScript 2 is deleted as its caching time is less than T_t ; and the number of leaves (i.e., correlated websites) in the trimmed tree is 5. Finally, we count during our measurements how many websites have at least N_t leaves (N_t is called the scale threshold) in their trimmed Website Correlation Trees given the time threshold T_t .

We visualize the number of websites that have at least N_t leaves in their trimmed Website Correlation Tree with time threshold T_t in heatmaps. In Figure 21, we choose T_t to be the local caching time threshold. For example, there are 380,994 websites given the scale threshold of 5,000

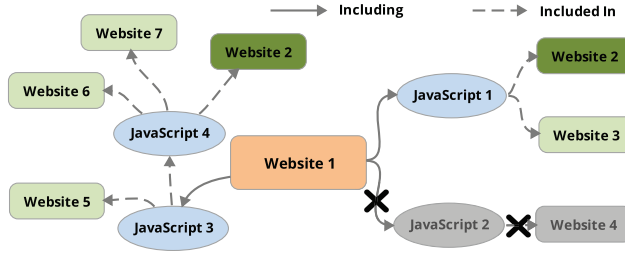


Fig. 20. Example of trimmed website correlation tree with Website 1 as the root.

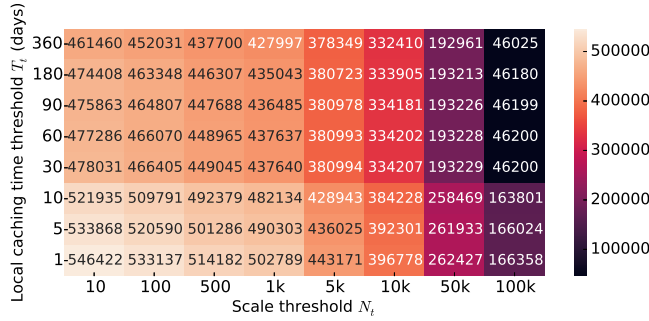


Fig. 21. Number of websites given local caching time threshold T_t and scale threshold N_t .

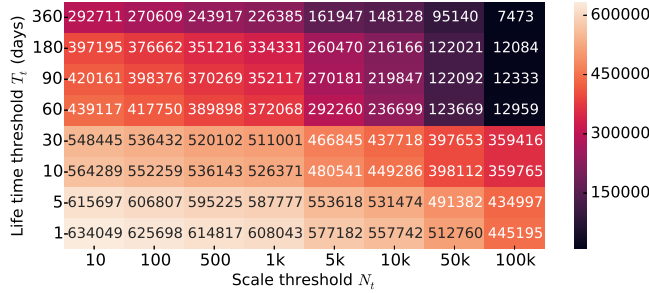


Fig. 22. Number of websites given life time threshold T_t and scale threshold N_t .

websites and time threshold of 30-day local caching time. Figure 22 shows the similar results with T_t being the life time threshold. Note that we consider lower bound of the life time to give a conservative estimation. For example, there are 466,845 websites for the same scale and time thresholds (i.e., 5,000 correlated websites and 30-day life time). The results show that there are a significant number of websites that are massively correlated with other websites and at the same time include third-party scripts with long caching time.

6.6 Validation with Updated Measurements

To evaluate whether the findings of our study remain applicable in the current web ecosystem, we conducted an updated measurement using a recently collected dataset in February and March 2026. Specifically, we performed a partial replication on a stratified sample of 100,000 websites drawn from the original list of one million websites, using the same crawling framework and analysis pipeline

Table 4. Comparison Between the Original Dataset and the Updated Dataset

Metric	Original Dataset	Updated Dataset
Websites including third-party scripts	84.92%	86.56%
Average third-party scripts per website	8.62	9.21
Average websites sharing the same third-party script	25.42	25.62
Maximum websites including one script	504,692	615,356
Websites including scripts with local caching time > 180 days	57.58%	60.25%
Websites including scripts with life time > 180 days	54.04%	53.69%
Websites given scale and local caching time thresholds of 5,000 and 30	39.25%	42.86%
Websites given scale and life time thresholds of 5,000 and 30	48.09%	46.78%

The results show that key metrics remain highly consistent across both datasets, confirming that the structural properties of third-party JavaScript inclusion and caching persist over time.

described in Section 5. Table 4 summarizes the comparison between the original dataset and the updated measurements. Overall, the results show strong consistency across all key metrics, with only minor variations observed. The insecure practices identified in our original analysis also persist in the updated dataset, reinforcing that the observed risks remain relevant in the current web ecosystem.

First, the prevalence of third-party JavaScript inclusion remains high, increasing slightly from 84.92% to 86.56%, while the average number of third-party scripts per website increases from 8.62 to 9.21. This indicates that modern websites continue to rely heavily on third-party JavaScript resources. Second, the structural properties of cross-site script reuse remain stable. The average number of websites sharing the same third-party script remains nearly unchanged (25.42 vs. 25.62), while the maximum reuse increases from 504,692 to 615,356 websites, primarily driven by widely deployed scripts such as those from Google Analytics. This suggests that highly shared scripts continue to dominate the ecosystem and may even become more centralized over time. Third, caching-related behaviors remain largely consistent. The fraction of websites including scripts with long local caching times (> 180 days) increases slightly (57.58% to 60.25%), while the fraction of scripts with long life time remains stable (54.04% vs. 53.69%). These results indicate that long-lived caching continues to be widely adopted. Finally, when considering combined scale and time thresholds, the results also remain consistent. For example, the fraction of websites including scripts that exceed both scale and local caching thresholds (5,000 sites and 30 days) increases moderately (39.25% to 42.86%), while the corresponding life time-based metric remains comparable (48.09% vs. 46.78%).

Overall, these results confirm that the key trends identified in our original study persist in the updated dataset. Despite the natural evolution of the web ecosystem, the structural risks associated with large-scale third-party JavaScript reuse and long-lived caching remain prevalent.

7 Discussion and Limitations

In this section, we discuss the limitations of our study and provide additional context for interpreting our findings. While our large-scale measurement reveals structural risks associated with third-party JavaScript inclusion and caching, our methodology involves several design choices and assumptions that may affect the precision of the results. We therefore clarify the scope and applicability of our analysis, discuss potential sources of measurement error, and examine how modern web security mechanisms influence the feasibility of the attack scenarios considered in this work. These discussions provide a more balanced and transparent understanding of the strengths and limitations of our approach.

7.1 Host List-Based Identification

Our study relies on a host list-based approach to identify third-party JavaScript resources. While this method effectively captures widely shared script providers, it may introduce classification

errors in certain cases. For example, some domains may host both site-specific and shared resources, which can lead to false positives, while less frequently used or newly deployed third-party services may not be included in the host list, resulting in false negatives.

The host list is constructed using a reference-frequency threshold (i.e., requiring a domain to be referenced by at least two distinct websites), which is designed to capture hosts shared across multiple websites. This aligns with the primary goal of our analysis, which is to characterize cross-site JavaScript reuse. Scripts referenced by only a single website do not contribute to cross-site connections and therefore have limited impact on our measurements, even if they are not captured by the host list. Furthermore, our analysis focuses on large-scale inclusion patterns driven by highly referenced scripts, which dominate the observed ecosystem. As a result, the key findings are robust to reasonable variations in the classification threshold, since low-frequency domains contribute minimally to the overall trends.

7.2 Interpretation of Caching Behavior

The analysis of caching behavior in this work uses the “10% rule” as a heuristic baseline for interpreting cache lifetime settings rather than as a strict requirement. While long caching durations are commonly adopted and can be appropriate for stable, versioned resources (e.g., popular libraries served via CDNs), they may introduce security risks when applied to non-versioned or dynamically changing scripts. In such cases, stale or compromised content may persist in the browser cache without timely invalidation.

We also note that the cache lifetimes measured in this study reflect the maximum persistence permitted by server-side caching policies. In practice, the actual lifetime of cached scripts may be shorter due to browser cache eviction caused by storage pressure, software updates, or user actions. Therefore, our findings should be interpreted as identifying conditions that may amplify security risks, rather than categorizing all long-lived caching configurations as inherently insecure.

7.3 Modern Web Security Environment

Modern web security mechanisms, including the widespread deployment of HTTPS and HSTS, significantly reduce the feasibility of in-transit script injection attacks. HTTPS protects scripts against modification during transmission, while HSTS further prevents protocol downgrade and insecure HTTP access after an HSTS policy has been established. Consequently, the MITM-based scenario considered in this work is substantially less feasible when these mechanisms are correctly deployed. However, the threat model considered in this work is not limited to MITM attacks. Malicious scripts may also be introduced through compromised third-party providers, CDN infrastructure, or software supply-chain attacks. HTTPS and HSTS do not prevent such attacks because the malicious content may be delivered through an otherwise authenticated and encrypted connection. Once introduced, caching may extend the persistence of the compromised script within the affected caching context.

Modern browsers have also introduced defenses such as cache partitioning, in which cached resources are isolated according to the top-level website. This mechanism directly reduces cross-site cache reuse: a malicious script cached while visiting one website will generally not be reused solely because the user later visits an unrelated website that references the same script URL. Therefore, cache partitioning can substantially limit the cross-site “revival” effect described in this work. Nevertheless, it does not prevent repeated reuse of a compromised cached script within the same website context, nor does it prevent multiple websites from independently retrieving a malicious script while a shared third-party provider or software dependency remains compromised. Moreover, legacy browsers, embedded webviews, and configurations without equivalent cache partitioning may remain exposed to cross-site cache reuse. Accordingly, our measurements should be interpreted

as characterizing potential risk amplification and shared-dependency concentration, rather than the number of attacks that would necessarily succeed under every modern browser configuration.

7.4 Lifetime Estimation and Bounding Accuracy

The lifetime of JavaScript resources in this study is estimated using upper and lower bounds derived from discrete crawling snapshots. Due to the weekly crawling granularity, multiple updates to a script may occur between consecutive crawls and are not directly observable. As a result, the estimated upper bound may overestimate the true lifetime, while the lower bound may underestimate it in certain cases. Therefore, the reported lifetime metrics should be interpreted as bounded approximations rather than exact measurements of script persistence. Despite this limitation, the analysis captures large-scale trends in caching behavior, as scripts with long effective lifetimes typically persist across multiple consecutive crawls.

7.5 Measurement Limitations and Practical Considerations

Our measurement is based on large-scale web crawling and therefore has inherent limitations. First, the crawling process focuses primarily on homepage content and may miss scripts that are conditionally loaded or triggered by user interaction. Second, the weekly crawling granularity may not capture all intermediate script updates, which may affect the precision of lifetime estimation. Third, browser cache behavior in practice may differ from theoretical cache lifetimes due to eviction policies, storage constraints, or user actions. In addition, our dataset is derived from popular websites, which may not fully represent smaller or less frequently visited sites in the broader web ecosystem. Finally, while the measurement reveals structural risks associated with third-party JavaScript reuse and caching, it does not directly quantify the likelihood of real-world exploitation. These limitations highlight that our results identify potential risk amplification factors rather than guaranteed attack outcomes.

8 Improving Security of Third-Party JavaScript Caching and Discussions

The measurement results have identified that there are a number of insecure practices on the Internet that overlook or even increase the security risk of third-party JavaScript inclusion and caching. To minimize such a risk, users may reduce the persistent storage of cached content by using private-browsing modes (e.g., Incognito mode in Chrome) or clearing their browser caches. However, caching web content is still the default configuration for most modern web browsers. Moreover, an ordinary user is not expected to be well aware of any security risk and choose his/her caching setups accordingly. Hence, we discuss potential methods to be used by third-party service providers, websites and browser developers to minimize the adverse impact of third-party JavaScript inclusion and caching.

- *Simplifying Inclusion Relationships and Proper Localization.* We find that the height of the JavaScript Inclusion Tree of a third-party script can have a large value, with the maximum being 18. This reveals that JavaScript inclusion relationships can be quite complicated on today's Internet, and thus website developers may be confused and overlook the inclusion relationships in their designs. Third-party JavaScript developers should simplify the inclusion relationships as much as possible. It's also essential for website developers to regularly update the JavaScript Inclusion Tree and store all third-party scripts at local servers to minimize the security risk.
- *Adequate Configurations of Caching Settings.* Our results show that third-party service providers often configure caching parameters such as Expires and max-age with long lifetimes. While such configurations are common for stable, versioned resources (e.g., libraries served via CDNs) and can improve performance, they rely on proper versioning and update

mechanisms to ensure content freshness. When these assumptions do not hold, such as for non-versioned or dynamically changing scripts, long cache lifetimes may introduce security risks by allowing stale or compromised content to persist.

We therefore recommend carefully aligning caching policies with the update characteristics of the script. For dynamically changing or security-sensitive scripts, conservative cache lifetimes or mechanisms such as revalidation should be adopted. In addition, websites handling sensitive information may consider enabling restrictive directives in Cache-Control to limit caching behavior. For example, no-cache enforces revalidation before reuse, while no-store disables local caching entirely.

- *Timely Updating Third-party Scripts.* Our measurement reveals that a considerable number of third-party scripts are rarely maintained and have not been updated for years, allowing an attack to survive in the cache for quite a while. We suggest that web developers should regularly check the included third-party JavaScript libraries, and keep them up-to-date. Meanwhile, third-party providers may also maintain a routine update to avoid a script being cached for too long.
- *Isolated Caching.* As discussed, if a widely-included third-party script has been manipulated by an attacker, it may potentially infect all the websites including this malicious script. We suggest that caching scripts for different websites can be isolated from each other, thus containing a potentially malicious script in the scope of a single website. Specifically, developers may make each third-party script dedicated to a particular website. This can be achieved by (i) giving a unique path or file name of the script (by third-party providers) to a particular website, (ii) appending a query string to the URL of the third-party script, or (iii) setting proper cache control parameters in the HTTP header. This practice ensures that the script will be treated as a unique entity for the caching purpose at a user's browser, thereby minimizing the adverse impact of third-party JavaScript inclusion and caching. Such approaches may increase bandwidth consumption and reduce CDN efficiency, highlighting a tradeoff between performance and security.
- *No Caching over Untrustworthy Connections.* A user may face MITM attacks when connecting to an untrustworthy public network. We suggest that a browser may never keep any cached files over the untrustworthy connections. Especially when a user visits a website via clicking through an HTTPS certificate warning, the browser can minimize the security risk by disabling caching at the moment that it issues the HTTPS certificate warning. The browser can resume users' caching setting (or even clean all existing cached contents) when it restarts. The latest version of Chrome has featured the function of disabling caching when a user bypasses the HTTPS certificate warning. We believe that such a feature should be adopted by all other browsers to minimize the security risk of caching a malicious script. The effectiveness of such browser-side mechanisms depends on deployment conditions and is discussed qualitatively in this work.
- *Integrity Verification with SRI.* SRI is a mechanism that allows developers to specify a cryptographic hash (e.g., SHA-256 and SHA-384) for external scripts, ensuring that only content matching the expected hash is executed by the browser. This mechanism can effectively prevent the execution of modified or malicious scripts, including those served from cache. However, our measurements (Section 6.2) show that SRI adoption remains low in practice. This is partly due to operational challenges, such as the need to update hash values when scripts change, as well as limited applicability in dynamic scenarios where script content varies frequently (e.g., advertising or analytics scripts). While SRI is effective for stable, versioned resources, its limited deployment in real-world settings highlights practical barriers to adoption and the need for improved tooling and automation.

- *Content Security Policy (CSP)*. CSP is a browser-enforced mechanism that enables websites to restrict the origins from which scripts and other resources can be loaded. It is effective in mitigating risks such as XSS and unauthorized third-party inclusion. However, its deployment in practice is often limited by configuration complexity and compatibility constraints. As a result, many deployed policies are overly permissive or only partially enforced, which reduces their effectiveness in mitigating real-world risks.
- *Sandboxing Mechanisms*. This technique aims to isolate untrusted or potentially malicious scripts by restricting their access to sensitive APIs or browser features. Common approaches include loading third party content in isolated iframes with restrictive attributes (e.g., sandbox and allow-scripts) and using browser features like site isolation. While sandboxing can reduce the impact of compromised scripts, it is not widely adopted due to compatibility concerns and limitations in functionality for embedded content. As a result, its practical effectiveness remains constrained in real-world deployments.
- *Trusted Script Hash Verification by Browsers*. An interesting future direction is to explore browser-level mechanisms for proactively validating widely used third party scripts. As observed in our study (e.g., Tables 1 and 2), a small number of third party scripts are included by a large portion of websites. Given their popularity and stability, browsers could maintain a trusted hash list for these scripts and verify their integrity before execution. This approach could offer a scalable and transparent defense against cache poisoning and script injection, even in the absence of developer-configured protections such as SRI. While such functionality may require coordination between browser vendors, content providers, and standardization bodies, it presents a promising opportunity to improve the baseline security of web ecosystems.

In practice, these mitigation strategies involve trade-offs among security, performance, compatibility, and maintenance effort. Disabling caching or requiring frequent revalidation can reduce the persistence of compromised scripts, but may increase network traffic and page-loading latency. Isolated caching and script localization can limit cross-site exposure, but may reduce CDN and shared-cache efficiency and require website operators to maintain and promptly update local copies. SRI provides strong integrity protection for stable scripts, but requires hash updates whenever script content changes and is less suitable for dynamic resources. Similarly, restrictive CSP and sandboxing policies can limit the capabilities of compromised scripts, but may require substantial configuration effort and can affect existing functionality. Therefore, these mechanisms should be selected and combined according to the update frequency, functionality, and security sensitivity of the deployed scripts.

9 Related Work

In this section, we summarize the literature related to our research in this article.

- *Malicious JavaScript Injection Strategies*. Our study is motivated by the security concern that when a cached JavaScript file is compromised, access to any website including the same script can be vulnerable. Our work is related to existing works [16–22] that detail the attack strategies to compromise a JavaScript file and inject the malicious script into a web browser, such as MITM based manipulations [17], the Pretty-Bad-Proxy attack [16], and modified HTTPS certificate attack [18]. Recently, the work [41] explores evasive script behaviors and dynamic deletion patterns. This aligns with our broader motivation to understand real-world practices that can hinder forensic analysis and risk mitigation. Our research does not focus on designing an attack strategy that has been well studied in the literature; rather, we aim to assess the scope and security indication of third-party JavaScript inclusion and caching on the Internet.

- *Web Cache Poisoning*. The article [42] presents a large-scale empirical study of cache poisoning attacks across real-world websites. It identifies how attackers can exploit inconsistencies in HTTP caching behavior to serve malicious content to unsuspecting users by manipulating cache entries. The study demonstrates the prevalence and persistence of such attacks in practice and highlights the challenges in detecting and mitigating them at scale. Our work complements this study by focusing more narrowly on the risks introduced by cached third party JavaScript, which inherits similar persistence characteristics but operates under different trust and inclusion models. Rather than examining generic cache poisoning vectors, we analyze how long-lived third party scripts, once compromised or injected, can silently persist in browser caches across a wide set of websites due to shared dependencies and insufficient cache validation. In contrast to their endpoint-focused attack model, our study emphasizes the ecosystem-level amplification caused by shared script URLs, browser reuse behavior, and correlated inclusion patterns across websites.
- *Large-scale Measurements for Security Analysis*. Network measurement studies have been conducted in a line of works [43–50] to perform the security analysis for the Internet from various perspectives. For example, ZMap [48] developed a fast Internet-wide scanning tool, which has enabled a wide range of security applications, such as botnet analysis [51], **domain name system (DNS)** manipulation analysis [51], and monitoring of global censorship [52]. The research in this article focuses on analyzing the current practice of third-party JavaScript inclusion and caching, and identifying the associated security risk. Our research is complementary to these works.
- *Large-scale JavaScript Measurements on the Internet*. Our work is also related to prior measurement studies of the client-side JavaScript libraries on the Internet. For example, the work in [53] quantified the reliability of third-party JavaScript providers based on Alexa top-10 thousand websites, and discovered potential attacks against such providers. The work [54] systematically explores the web’s implicit trust chains and highlights how indirect inclusions can introduce security risks. In [55], the authors measured the usage of outdated JavaScript libraries over 133,000 websites, and revealed that a large number of websites still use outdated JavaScript libraries or libraries with known vulnerabilities. In contrast, the scale of our measurement is larger and we point out the insecure practices about JavaScript inclusion and caching in today’s websites and third-party service providers.
 There also exist a number of papers [6, 56–58] focusing on measurement studies about potentially vulnerable use of specific JavaScript functions. For example, the insecure usage of JavaScript’s `localStorage()` function among Alexa top-500,000 websites was studied in [56] for code caching purposes. In [57], the authors performed a measurement study to identify insecure practices of script inclusion and dynamic generation (e.g., usage of functions `eval()` or `document.write()`) over 6,805 unique websites. An empirical study in [25] aimed to understand different privacy violating information flows raised by the dynamic generation nature of JavaScript. Our work does not focus on specific JavaScript vulnerabilities; rather, we conduct a study to understand the network and caching related metrics regarding the JavaScript deployment on the Internet, and to quantify the security risk of third-party JavaScript inclusion and caching.
- *AI-driven Security Analysis Towards Web Security*. Recent advances in AI-driven security analysis have created new opportunities for automating the detection of JavaScript-based vulnerabilities and anomalous behaviors in web resource loading. Prior work has explored the use of machine learning techniques to analyze JavaScript code, detect obfuscated or malicious scripts through static and dynamic features, and identify deviations in inclusion and caching behavior [59, 60]. These approaches extend beyond traditional rule-based systems by

leveraging statistical models and deep learning techniques that can generalize to previously unseen threats. While our current study focuses on large-scale empirical measurements, the inclusion correlation structures and caching patterns we reveal may serve as valuable features for future ML-based detection systems. We view the integration of AI techniques with empirical web security research as a promising direction to further enhance the robustness and scalability of automated threat detection.

- *Browser-level Defenses*. Recent browser-level defenses, such as site isolation and cache partitioning (e.g., double-keyed caching), aim to mitigate cross-site resource sharing risks by enforcing stronger isolation across origins and top-level sites [61–63]. These mechanisms represent an important architectural direction in modern browser security to reduce unintended sharing of resources and state. Our work is complementary to these approaches. While browser-level isolation limits cross-site interactions at the platform level, we focus on measuring structural risks arising from large-scale third-party JavaScript inclusion and caching practices in real-world deployments, which remain prevalent due to heterogeneous browser support and deployment configurations.

10 Conclusions

In this article, we present a comprehensive study to analyze how current practices of third-party JavaScript deployment and caching may exacerbate the adverse impact of malicious scripts. We evaluate the security risk caused by third-party JavaScript inclusion and caching from both scale and time perspectives, and identify corresponding insecure practices on the Internet. We also discuss potential solutions to minimize the security risk. Our study is instrumental in helping website developers to improve the security perspectives of third-party JavaScript management.

References

- [1] Tao Hou, Shengping Bi, Mingkui Wei, Tao Wang, Zhuo Lu, and Yao Liu. 2022. When third-party javascript meets cache: Explosively amplifying security risks on the internet. In *Proc. of IEEE Conference on Communications and Network Security*.
- [2] *jQuery*. Retrieved from <https://jquery.com>
- [3] *Google Analytics*. Retrieved from <https://developers.google.com/analytics/>
- [4] Adam Barth. 2011. The Web Origin Concept (RFC 6454). Internet Engineering Task Force. Retrieved from <https://www.rfc-editor.org/rfc/rfc6454.html>
- [5] Philipp Vogt, Florian Nentwich, Nenad Jovanovic, Engin Kirda, Christopher Kruegel, and Giovanni Vigna. 2007. Cross site scripting prevention with dynamic data tainting and static analysis. In *Proc. of NDSS 2007* (2007), 12.
- [6] Sebastian Lekies, Ben Stock, and Martin Johns. 2013. 25 million flows later: Large-scale detection of DOM-based XSS. In *Proc. of ACM CCS*. 1193–1204.
- [7] Adam Barth, Collin Jackson, and John C. Mitchell. 2008. Robust defenses for cross-site request forgery. In *Proc. of ACM CCS*. 75–88.
- [8] Sebastian Lekies, Ben Stock, Martin Wentzel, and Martin Johns. 2015. The unexpected dangers of dynamic JavaScript.. In *Proc. of USENIX Security*. 723–735.
- [9] Yaoqi Jia, Yue Chen, Xinshu Dong, Prateek Saxena, Jian Mao, and Zhenkai Liang. 2015. Man-in-the-browser-cache: Persisting HTTPS attacks via browser cache poisoning. *Computers & Security* 55 (2015), 62–80.
- [10] Tobias Lauinger, Abdelberi Chaabane, Ahmet Salih Buyukkayhan, Kaan Onarlioglu, and William Robertson. 2017. Game of registrars: An empirical analysis of post-expiration domain name takeovers. In *Proc. of USENIX Security*. 865–880.
- [11] Chen Wang, Xiuyuan Zheng, Yingying Jennifer Chen, and Jie Yang. 2017. Locating rogue access point using fine-grained channel information. *IEEE Transactions on Mobile Computing* 16, 9 (2017), 2560–2573.
- [12] Amit Levy, Henry Corrigan-Gibbs, and Dan Boneh. 2016. Stickler: Defending against malicious content distribution networks in an unmodified browser. *IEEE Security & Privacy* 14 (2016).
- [13] Sean-Philip Oriyano and Robert Shemonski. 2012. *Client-side Attacks and Defense*. Newnes.
- [14] *The Chromium Projects*. Retrieved from <https://www.chromium.org/>

- [15] R. Fielding, Mark Nottingham, and J. Reschke. 2014. Hypertext transfer protocol (HTTP/1.1): Caching (RFC 7234). Internet Engineering Task Force. Retrieved from <https://www.rfc-editor.org/rfc/rfc7234.html>
- [16] Shuo Chen, Ziqing Mao, Yi-Min Wang, and Ming Zhang. 2009. Pretty-bad-proxy: An overlooked adversary in browsers' HTTPS deployments. In *Proc. of IEEE S&P*. 347–359.
- [17] OWASP Foundation. 2009. Man-in-the-browser attack. Retrieved from https://owasp.org/www-community/attacks/Man-in-the-browser_attack
- [18] Franco Callegati, Walter Cerroni, and Marco Ramilli. 2009. Man-in-the-Middle Attack to the HTTPS Protocol. *IEEE Security & Privacy* 7, 1 (2009), 78–81.
- [19] Moxie Marlinspike. 2009. More tricks for defeating SSL in practice. In *Proc. of Black Hat USA*.
- [20] Karthikeyan Bhargavan, Antoine Delignat Lavaud, Cédric Fournet, Alfredo Pironti, and Pierre Yves Strub. 2014. Triple handshakes and cookie cutters: Breaking and fixing authentication over TLS. In *Proc. of IEEE S&P*. 98–113.
- [21] Sruthi Bandhakavi, Samuel T. King, Parthasarathy Madhusudan, and Marianne Winslett. 2010. VEX: Vetting browser extensions for security vulnerabilities.. In *Proc. of USENIX Security*, Vol. 10. 339–354.
- [22] Adam Barth, Adrienne Porter Felt, Prateek Saxena, and Aaron Boodman. 2010. Protecting browsers from extension vulnerabilities. In *Proc. of NDSS*.
- [23] Italo Dacosta, Saurabh Chakradeo, Mustaque Ahamad, and Patrick Traynor. 2012. One-time cookies: Preventing session hijacking attacks with stateless authentication tokens. *ACM Transactions on Internet Technology* 12, 1, Article No 1 (2012), 1–24. DOI: <https://doi.org/10.1145/2220352.2220353>
- [24] Artur Janc. 2011. Rootkits in your web application. In *Proc. of 28C3: Chaos Communications Congress*. Berlin, Germany.
- [25] Dongseok Jang, Ranjit Jhala, Sorin Lerner, and Hovav Shacham. 2010. An empirical study of privacy-violating information flows in JavaScript web applications. In *Proc. of ACM CCS*. 270–283.
- [26] Zhou Li, Kehuan Zhang, Yinglian Xie, Fang Yu, and XiaoFeng Wang. 2012. Knowing your enemy: Understanding and detecting malicious web advertising. In *Proc. of ACM CCS*. 674–686.
- [27] Kaimin Zhang, Lu Wang, Aimin Pan, and Bin Benjamin Zhu. 2010. Smart caching for web browsers. In *Proc. of WWW*. 491–500.
- [28] Massimo Guarascio, Marco Zuppelli, Nunziato Cassavia, Luca Caviglione, and Giuseppe Manco. 2022. Revealing MageCart-like threats in favicons via artificial intelligence. In *Proc. of 17th International Conference on Availability, Reliability and Security*. 1–7.
- [29] Jens Frieß, Tobias Gattermayer, Nethanel Gelernter, Haya Schulmann, and Michael Waidner. 2024. Cloudy with a chance of cyberattacks: dangling resources abuse on cloud platforms. In *Proc. of 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1977–1994.
- [30] Xinyi Zheng, Chen Wei, Shenao Wang, Yanjie Zhao, Peiming Gao, Yuanhao Zhang, Kailong Wang, and Haoyu Wang. 2024. Towards robust detection of open source software supply chain poisoning attacks in industry environments. In *Proc. of 39th IEEE/ACM International Conference on Automated Software Engineering*. 1990–2001.
- [31] Devdatta Akhawe and Adrienne Porter Felt. 2013. Alice in Warningland: A large-scale field study of browser security warning effectiveness. In *Proc. of USENIX Security*, Vol. 13.
- [32] Rachna Dhamija, J. Doug Tygar, and Marti Hearst. 2006. Why phishing works. In *Proc. of ACM CHI*. 581–590.
- [33] Joshua Sunshine, Serge Egelman, Hazim Almuhtedi, Neha Atri, and Lorrie Faith Cranor. 2009. Crying Wolf: An empirical study of SSL warning effectiveness. In *Proc. of USENIX Security*. 399–416.
- [34] Jeremy Clark and Paul C. van Oorschot. 2013. SoK: SSL and HTTPS: Revisiting past challenges and evaluating certificate trust model enhancements. In *Proc. of IEEE S&P*. 511–525.
- [35] *Chrome DevTools Protocol*. Retrieved from <https://chromedevtools.github.io/devtools-protocol>
- [36] *Headless Chromium*. Retrieved from <https://chromium.googlesource.com/chromium/src/+lkgr/headless/README.md>
- [37] Eric Bidelman. 2017. *Getting Started with Headless Chrome*. Retrieved from <https://developers.google.com/web/updates/2017/04/headless-chrome>
- [38] Eric Bidelman. 2017. *Automated Testing with Headless Chrome*. Retrieved from <https://developers.google.com/web/updates/2017/06/headless-karma-mocha-chai>
- [39] W3C. 2012. *HTTP Archive (HAR) Format*. Retrieved from <https://dvcs.w3.org/hg/webperf/raw-file/tip/specs/HAR/Overview.html>
- [40] *Google Hosted Libraries*. Retrieved from <https://developers.google.com/speed/libraries/>
- [41] Xinzhe Wang, Zeyang Zhuang, Wei Meng, and James Cheng. 2024. Detecting and understanding self-deleting javascript code. In *Proc. of ACM Web Conference 2024*. 1768–1778.
- [42] Yuejia Liang, Jianjun Chen, Run Guo, Kaiwen Shen, Hui Jiang, Man Hou, Yue Yu, and Haixin Duan. 2024. Internet's invisible enemy: Detecting and measuring web cache poisoning in the wild. In *Proc. of 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 452–466.
- [43] Zakir Durumeric, James Kasten, David Adrian, J. Alex Halderman, Michael Bailey, Frank Li, Nicolas Weaver, Johanna Amann, Jethro Beekman, Mathias Payer, et al. 2014. The matter of heartbleed. In *Proc. of ACM IMC*. 475–488.

- [44] Zakir Durumeric, James Kasten, Michael Bailey, and J. Alex Halderman. 2013. Analysis of the HTTPS certificate ecosystem. In *Proc. of ACM IMC*. 291–304.
- [45] Zain Shamsi, Ankur Nandwani, Derek Leonard, and Dmitri Loguinov. 2014. Hershel: Single-packet OS fingerprinting. In *Proc. of ACM SIGMETRICS Performance Evaluation Review*, Vol. 42. 195–206.
- [46] Matthew Luckie, Robert Beverly, Tiange Wu, Mark Allman, and Kc Claire Claffy. 2015. Resilience of deployed TCP to blind attacks. In *Proc. of ACM IMC*. 13–26.
- [47] Scott Yilek, Eric Rescorla, Hovav Shacham, Brandon Enright, and Stefan Savage. 2009. When private keys are public: Results from the 2008 Debian OpenSSL vulnerability. In *Proc. of ACM IMC*. 15–27.
- [48] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. 2013. ZMap: Fast internet-wide scanning and its security applications. In *Proc. of USENIX Security*, Vol. 8. 47–53.
- [49] Alan Quach, Zhongjie Wang, and Zhiyun Qian. 2017. Investigation of the 2016 Linux TCP Stack Vulnerability at Scale. In *Proc. of ACM SIGMETRICS*.
- [50] Manos Antonakakis, Tim April, Michael Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, et al. 2017. Understanding the mirai botnet. In *Proc. of USENIX Security*.
- [51] Paul Pearce, Ben Jones, Frank Li, Roya Ensafi, Nick Feamster, Nick Weaver, and Vern Paxson. 2017. Global measurement of DNS manipulation. In *Proc. of USENIX Security*.
- [52] Paul Pearce, Roya Ensafi, Frank Li, Nick Feamster, and Vern Paxson. 2017. Augur: Internet-Wide Detection of Connectivity Disruptions. In *Proc. of IEEE S&P*. 427–443.
- [53] Nick Nikiforakis, Luca Invernizzi, Alexandros Kapravelos, Steven Van Acker, Wouter Joosen, Christopher Kruegel, Frank Piessens, and Giovanni Vigna. 2012. You are what you include: Large-scale evaluation of remote JavaScript inclusions. In *Proc. of ACM CCS*. 736–747.
- [54] Muhammad Ikram, Rahat Masood, Gareth Tyson, Mohamed Ali Kaafar, Noha Loizon, and Roya Ensafi. 2019. The chain of implicit trust: An analysis of the web third-party resources loading. In *Proc. of World Wide Web Conference*. 2851–2857.
- [55] Tobias Lauinger, Abdelberi Chaabane, Sajjad Arshad, William Robertson, Christo Wilson, and Engin Kirda. 2017. Thou shalt not depend on me: Analysing the use of outdated javascript libraries on the web. In *Proc. of NDSS*.
- [56] Sebastian Lekies and Martin Johns. 2012. Lightweight integrity protection for web storage-driven content caching. In *Proc. of 6th Workshop on Web*, Vol. 2.
- [57] Chuan Yue and Haining Wang. 2009. Characterizing insecure JavaScript practices on the web. In *Proc. of WWW*. 961–970.
- [58] Gregor Richards, Christian Hammer, Brian Burg, and Jan Vitek. 2011. The eval that men do. *ECOOP 2011 – Object-Oriented Programming*, M. Mezini (Eds.). Vol. 6813, Lecture notes in computer science, Springer, Berlin, Heidelberg. DOI: https://doi.org/10.1007/978-3-642-22655-7_4
- [59] Yueming Wu, Kunlun Ren, Weizhong Qiang, Yi Zhou, Deqing Zou, and Hai Jin. 2023. JSRevealer: A robust malicious javascript detector against obfuscation. In *Proc. of 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–12.
- [60] Marius Musch and Martin Johns. 2021. U can’t debug this: Detecting javascript anti-debugging techniques in the wild. In *30th USENIX Security Symposium (USENIX Security 21)*. 2935–2950.
- [61] Charles Reis, Alexander Moshchuk, and Nasko Oskov. 2019. Site isolation: Process separation for web sites within the browser. In *28th USENIX Security Symposium (USENIX Security 19)*. 1661–1678.
- [62] Shravan Narayan, Craig Disselkoen, Tal Garfinkel, Nathan Froyd, Eric Rahm, Sorin Lerner, Hovav Shacham, and Deian Stefan. 2020. Retrofitting fine grain isolation in the Firefox renderer. In *29th USENIX Security Symposium (USENIX Security 20)*. 699–716.
- [63] Qi Zhang, Qian Wu, Zeqi Lai, Jihao Li, Hewu Li, Yuyu Liu, Yuanjie Li, and Jun Liu. 2025. Spache: Accelerating ubiquitous web browsing via schedule-driven space caching. In *Proc. of ACM on Web Conference 2025*. 740–750.

Received 30 September 2024; revised 18 March 2026; accepted 11 July 2026